Q-1.What is SQL, and why is it essential in database management?

A.SQL (Structured Query Language) is a standard programming language used to create, manage, and manipulate data in relational database management systems (RDBMS).

1. **Data Retrieval**

- SQL allows users to fetch data using the `SELECT` statement.

2. **Data Manipulation**

- Insert, update, and delete records using `INSERT`, `UPDATE`, and `DELETE`.

3. **Database Structure Management**

- Create and modify tables using `CREATE`, `ALTER`, and `DROP`.

4. **Data Integrity**

- Maintains accuracy using constraints like `PRIMARY KEY`, `FOREIGN KEY`, and `UNIQUE`.

5. **Security & Access Control**

- Controls user permissions using `GRANT` and `REVOKE`.

6. **Transaction Management**

- Ensures safe database operations using `COMMIT` and `ROLLBACK`.

Q-2. Explain the difference between DBMS and RDBMS.

A.

Basis	DBMS	RDBMS
Full Form	Database Management System	Relational Database Management System

Data Storage	Stores data in files	Stores data in tables (rows & columns)
Relationship	Does not support relationships between tables	Supports relationships using keys
Normalization	Does not support normalization	Supports normalization
Security	Less secure	More secure
Multi-user Support	Limited multi-user support	Supports multiple users
Examples	dBase, FoxPro	MySQL, Oracle, SQL Server

Q-3. Describe the role of SQL in managing relational databases.

A.

SQL (Structured Query Language) plays a central role in managing relational databases because it allows users to create, access, and control data stored in tables.

1. Data Definition (DDL)

SQL is used to create and modify database structures.

- CREATE – create tables and databases
- ALTER – modify table structure
- DROP – delete tables

2. Data Manipulation (DML)

SQL manages the data inside tables.

- INSERT – add new records
- UPDATE – modify existing records
- DELETE – remove records

3. Data Retrieval (DQL)

- SELECT – fetch data from one or more tables
- Supports WHERE, JOIN, ORDER BY, GROUP BY

4. Relationship Management

- Maintains relationships using Primary Key and Foreign Key
- Ensures referential integrity

5. Security and Access Control (DCL)

- GRANT – give permissions
- REVOKE – remove permissions

6. Transaction Management (TCL)

- COMMIT – save changes
- ROLLBACK – undo changes

Q-4. What are the key features of SQL?

A.Key Features of SQL

1. Simple and Easy to Learn

- Uses simple English-like commands such as SELECT, INSERT, DELETE.

2. Data Definition Language (DDL)

- Create and modify database structure using CREATE, ALTER, DROP

3. Data Manipulation Language (DML)

- Insert, update, and delete records using INSERT, UPDATE, DELETE.

4. Data Query Language (DQL)

- Retrieve data using SELECT with conditions (WHERE, ORDER BY, GROUP BY).

5. Data Control Language (DCL)

- Provides security using GRANT and REVOKE.

6. Transaction Control Language (TCL)

- Maintains data consistency using COMMIT, ROLLBACK, and SAVEPOINT.

7. Supports Relationships

- Maintains relationships using Primary Key and Foreign Key.

8. Portability

- Works with major relational databases like MySQL, Oracle, PostgreSQL, and SQL Server.

Q-5. What are the basic components of SQL syntax?

A.Basic Components of SQL Syntax

1. Keywords

Reserved words that perform specific functions.
Examples: SELECT, INSERT, UPDATE, DELETE, CREATE, WHERE

2. Identifiers

Names of database objects such as:

- Table names
- Column names
- Database names

Example:

SELECT student_name FROM students;

Here, students and student_name are identifiers.

3. Clauses

Conditions or modifications used in a statement.
Examples:

- WHERE
- ORDER BY
- GROUP BY
- HAVING

4. Expressions

Combination of columns, values, and operators.

Example:

SELECT age + 1 FROM students;

5. Operators

Used to perform operations:

- Arithmetic (+, -, *)
- Comparison (=, >, <)
- Logical (AND, OR, NOT)

6. Literals

Fixed values used in queries.

Example:

WHERE age = 15;

Here, 15 is a literal.

Q-6.Write the general structure of an SQL SELECT statement.

A.SELECT column_name(s)

FROM table_name

WHERE condition

GROUP BY column_name(s)

HAVING condition

ORDER BY column_name(s) ASC | DESC;

Q-7.Explain the role of clauses in SQL statements.

A.Role of Clauses in SQL Statements

Clauses in SQL are used to define specific conditions and operations within a query. They help control how data is selected, filtered, grouped, and sorted.

1. SELECT Clause

- Specifies the columns to retrieve.
- Determines what data is displayed.

Example:

SELECT student_name, age

2. FROM Clause

- Specifies the table from which data is retrieved.
- Determines where data comes from.

Example:

FROM students

3. WHERE Clause

- Filters records based on conditions.
- Determines which rows are selected.

Example:

WHERE age > 15

4. GROUP BY Clause

- Groups rows with similar values.
- Used with aggregate functions like COUNT, SUM, AVG.

5. HAVING Clause

- Filters grouped records.
- Used after GROUP BY.

6. ORDER BY Clause

- Sorts the result in ascending (ASC) or descending (DESC) order.

Q-8. What are constraints in SQL? List and explain the different types of constraints.

A.Constraints are rules applied to table columns to ensure accuracy, integrity, and reliability of data in a database.

They prevent invalid data from being inserted into tables.

Types of Constraints in SQL

1. NOT NULL

- Ensures a column cannot have a NULL value.
- The field must contain a value.

student_name VARCHAR(50) NOT NULL

2. UNIQUE

- Ensures all values in a column are different.
- No duplicate values allowed.

email VARCHAR(100) UNIQUE

3. PRIMARY KEY

- Uniquely identifies each record in a table.

- Cannot contain NULL values.
- Combination of NOT NULL + UNIQUE.

student_id INT PRIMARY KEY

4. FOREIGN KEY

- Establishes a relationship between two tables.
- References the PRIMARY KEY of another table.

FOREIGN KEY (teacher_id) REFERENCES teachers(teacher_id)

5. CHECK

- Ensures values meet a specific condition.

age INT CHECK (age >= 18)

6. DEFAULT

- Assigns a default value if no value is provided.

status VARCHAR(20) DEFAULT 'Active'

Q-9.How do PRIMARY KEY and FOREIGN KEY constraints differ?

A.

Basis	PRIMARY KEY	FOREIGN KEY
Purpose	Uniquely identifies each record in a table	Links one table to another
Uniqueness	Must be unique	Can have duplicate values

NULL Values	Cannot contain NULL	Can contain NULL (unless restricted)
Number per Table	Only one PRIMARY KEY per table	Multiple FOREIGN KEYS allowed
Role	Ensures entity integrity	Ensures referential integrity
Location	Defined in the parent table	Defined in the child table

Q-10.What is the role of NOT NULL and UNIQUE constraints?

A.1. NOT NULL Constraint

- Ensures that a column cannot contain NULL values.
- Every record must have a value in that column.
- Helps maintain data completeness.

Example:

student_name VARCHAR(50) NOT NULL

2. UNIQUE Constraint

- Ensures that all values in a column are different.
- Prevents duplicate entries
- Maintains data uniqueness.

Example:

email VARCHAR(100) UNIQUE

Q-11. Define the SQL Data Definition Language (DDL).

A.DDL commands are used to create, modify, and delete database structures.

Common DDL Commands

1. CREATE – Creates database objects

CREATE TABLE students ( );

2. ALTER – Modifies an existing table

ALTER TABLE students ADD age INT;

3. DROP – Deletes a table or database

DROP TABLE students;

4. TRUNCATE – Removes all records from a table (structure remains)

TRUNCATE TABLE students;

Q-12. Explain the CREATE command and its syntax.

A.The CREATE command is a DDL (Data Definition Language) statement used to create new database objects such as databases, tables, views, indexes, and procedures.

Syntax to Create a Database

CREATE DATABASE database_name;

Example:

CREATE DATABASE school_db;

Syntax to Create a Table

CREATE TABLE table_name (

    column1 datatype constraint,

    column2 datatype constraint,

```
    column3 datatype constraint
);
```

Example:

```
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(50) NOT NULL,
    age INT,
    address VARCHAR(100)
);
```

Q-13. What is the purpose of specifying data types and constraints during table creation?

A.When creating a table in SQL, data types and constraints are specified to ensure proper data storage, accuracy, and integrity.

Data types define **what kind of data** can be stored in a column.

Constraints apply rules to maintain **data integrity and consistency**.

Q-14. What is the use of the ALTER command in SQL?

A.The ALTER command is used to modify the structure of an existing table in a database.

Common Uses of ALTER

1. Add a new column

```
ALTER TABLE students
ADD email VARCHAR(100);
```

2. Modify a column

ALTER TABLE students

MODIFY age INT NOT NULL;

3. Drop a column

ALTER TABLE students

DROP address;

4.Add a constraint

ALTER TABLE students

ADD UNIQUE (email);

Q-15. How can you add, modify, and drop columns from a table using ALTER?

A.Using ALTER to Add, Modify, and Drop Columns

1. Add a Column

Syntax:

ALTER TABLE table_name

ADD column_name datatype;

Example:

ALTER TABLE students

ADD email VARCHAR(100);

2. Modify a Column

Syntax (MySQL / MariaDB):

ALTER TABLE table_name

MODIFY column_name new_datatype;

Example:

ALTER TABLE students

MODIFY age INT NOT NULL;

3. Drop a Column

Syntax:

ALTER TABLE table_name

DROP column_name;

Example:

ALTER TABLE students

DROP address;

Q-16. What is the function of the DROP command in SQL?

A.The DROP command is a DDL (Data Definition Language) statement used to permanently remove database objects such as tables, databases, views, indexes, or procedures.

Drop a Table

DROP TABLE students;

Drop a Database

DROP DATABASE school_db;

Q-17. What are the implications of dropping a table from a database?

A.1. Permanent loss of data
All records stored in the table are permanently deleted. The data cannot be recovered unless a backup exists.

2. Removal of table structure
The table definition (columns, data types, indexes, and constraints) is completely removed from the database.

3. Broken relationships
If other tables reference the dropped table through FOREIGN KEY constraints, those relationships are affected and may cause errors.

4. Impact on dependent objects
Views, stored procedures, triggers, or applications that use the table may fail or become invalid.

5. Auto-commit behavior
DROP TABLE is a DDL command and usually performs an implicit commit, so the action cannot be rolled back in most database systems.

Q-18.Define the INSERT, UPDATE, and DELETE commands in SQL

A.INSERT, UPDATE, and DELETE are SQL Data Manipulation Language (DML) commands used to modify data in database tables.

INSERT :

Definition:
The INSERT command is used to add new records (rows) into a table.

Purpose:
To store new data in the database.

Syntax:

INSERT INTO table_name (column1, column2, ...)

VALUES (value1, value2, ...);

Example:

INSERT INTO courses (course_id, course_name, course_duration)

VALUES (101, 'SQL Basics', '3 Months');

UPDATE :

Definition:
The UPDATE command is used to modify existing records in a table.

Purpose:
To change data that is already stored.

Syntax:

UPDATE table_name

SET column1 = value1, column2 = value2

WHERE condition;

Example:

UPDATE courses

SET course_duration = '4 Months'

WHERE course_id = 101;

DELETE :

Definition:
The DELETE command is used to remove existing records from a table.

Purpose:
To delete unwanted data.

Syntax:

DELETE FROM table_name

WHERE condition;

Example:

DELETE FROM courses

WHERE course_id = 101;

Q-19. What is the importance of the WHERE clause in UPDATE and DELETE operations?

A.1. Prevents accidental mass changes

- Without WHERE, the command affects all rows in the table.
- This can lead to serious data loss or unwanted updates.

Example (dangerous):

DELETE FROM courses;

Deletes every record in the table.

2. Targets specific records

- WHERE allows you to update or delete only the required rows based on a condition.

Example:

UPDATE courses

SET course_duration = '6 Months'

WHERE course_id = 102;

Only the matching course is updated.

3. Maintains data accuracy and integrity

- Ensures only intended data is modified.
- Helps avoid corruption of important data.

4. Improves control and safety

- Gives precise control over database operations.
- Essential in real-world applications where data is critical.

Q-20.What is the SELECT statement, and how is it used to query data?

A.SELECT is a Data Query Language (DQL) command that fetches data based on specified conditions.

How SELECT is used

1. Retrieve all columns

SELECT * FROM courses;

Displays all records and columns.

2. Retrieve specific columns

SELECT course_name, course_duration

FROM courses;

Shows only selected fields.

3. Filter data using WHERE

SELECT * FROM courses

WHERE course_duration = '3 Months';

Returns only matching rows.

4. Sort results using ORDER BY

SELECT * FROM courses

ORDER BY course_duration DESC;

5. Limit number of rows

SELECT * FROM courses

LIMIT 2;

Q-21. Explain the use of the ORDER BY and WHERE clauses in SQL queries.

A.ORDER BY Clause

Purpose:
The ORDER BY clause is used to sort the result set in ascending or descending order.

Key Points:

- Default order is ASC (ascending).
- Use DESC for descending order.
- Can sort by one or multiple columns.

Syntax:

SELECT column1, column2

FROM table_name

ORDER BY column_name ASC|DESC;

Example:

SELECT * FROM courses

ORDER BY course_duration DESC;

Displays courses sorted from highest to lowest duration.

WHERE Clause

Purpose:
The WHERE clause is used to filter records based on specified conditions. It ensures that only the rows meeting the condition are returned.

Key Points:

- Works like a condition (filter).
- Used with SELECT, UPDATE, and DELETE.
- Supports operators like =, >, <, LIKE, IN, BETWEEN, etc.

Syntax:

SELECT column1, column2

FROM table_name

WHERE condition;

Example:

SELECT * FROM courses

WHERE course_duration = '3 Months';

Returns only courses with duration of 3 months.

Q-22.What is the purpose of GRANT and REVOKE in SQL?

A.GRANT :

Purpose:
The GRANT command is used to give permissions to users or roles.

Common privileges:

- **SELECT**
- **INSERT**
- **UPDATE**
- **DELETE**
- **ALL PRIVILEGES**

Syntax:

GRANT privilege_name

ON table_name

TO user_name;

Example:

GRANT SELECT ON courses TO user1**;**

Allows user1 to read data from the courses table.

REVOKE :

Purpose:
The REVOKE command is used to remove previously granted permissions.

Syntax:

REVOKE privilege_name

ON table_name

FROM user_name;

Example:

REVOKE SELECT ON courses FROM user1;

Removes read permission from user1.

Q-23.How do you manage privileges using these commands?

A.Grant privileges

Use GRANT to give specific permissions.

Example — allow read access:

GRANT SELECT ON school_db.courses TO 'user1'@'localhost';

Example — multiple privileges:

GRANT SELECT, INSERT, UPDATE

ON school_db.courses

TO 'user2'@'localhost';

Example — all privileges:

GRANT ALL PRIVILEGES ON school_db.* TO 'user1'@'localhost';

Revoke privileges

Use REVOKE to remove permissions.

Example:

```
REVOKE INSERT, UPDATE

ON school_db.courses

FROM 'user2'@'localhost';
```

Q-24.What is the purpose of the COMMIT and ROLLBACK commands in SQL?

A.COMMIT

Purpose:
COMMIT permanently saves all changes made during the current transaction.

What it does:

- Makes changes permanent
- Ends the transaction
- Changes cannot be undone after commit

Syntax:

```
COMMIT;
```

Example:

```
UPDATE courses

SET course_duration = '6 Months'

WHERE course_id = 101;

COMMIT;
```

The update is permanently stored.

Q-25.Explain how transactions are managed in SQL databases.

A.1. Start the transaction

In many databases, a transaction starts automatically.
Or explicitly:

START TRANSACTION;

(or BEGIN;)

2. Execute SQL statements

Perform operations like INSERT, UPDATE, DELETE.

UPDATE accounts

SET balance = balance - 500

WHERE id = 1;

3. Save or undo

COMMIT — make changes permanent

COMMIT;

ROLLBACK — undo changes

ROLLBACK;

Q-26.Explain the concept of JOIN in SQL. What is the difference between INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN?

A.Concept of JOIN in SQL

A JOIN in SQL is used to combine rows from two or more tables based on a related column between them (usually a primary key and foreign key).

→ Joins help retrieve related data stored in different tables.

INNER JOIN

Meaning:
Returns only the rows where there is a **match in both tables**.

Key idea: Intersection of both tables.

Example:

```
SELECT students.student_name, courses.course_name

FROM students

INNER JOIN courses

ON students.course_id = courses.course_id;
```

Output:

Only matching records
Unmatched rows are excluded

LEFT JOIN (LEFT OUTER JOIN)

Meaning:
Returns **all rows from the left table** and matched rows from the right table.

If no match → NULL values from right table.

Key idea: Left table is fully preserved.

Example:

```
SELECT students.student_name, courses.course_name

FROM students

LEFT JOIN courses

ON students.course_id = courses.course_id;
```

Output:

All left table rows
Matching right rows
Unmatched right → NULL

RIGHT JOIN (RIGHT OUTER JOIN)

Meaning:
Returns **all rows from the right table** and matched rows from the left table.

If no match → NULL values from left table.

Key idea: Right table is fully preserved.

Example:

```
SELECT students.student_name, courses.course_name

FROM students

RIGHT JOIN courses

ON students.course_id = courses.course_id;
```

Output:

All right table rows
Matching left rows
Unmatched left → NULL

FULL OUTER JOIN

Meaning:
Returns **all rows from both tables**.

- Matching rows are combined
- Non-matching rows show NULLs

Key idea: Union of both tables.

Example:

```
SELECT students.student_name, courses.course_name

FROM students

FULL OUTER JOIN courses

ON students.course_id = courses.course_id;
```

Output:

All rows from both tables
Unmatched rows filled with NULL

Q-27.How are joins used to combine data from multiple tables?

A.In relational databases, data is stored in separate tables to reduce redundancy.

A JOIN is used to combine related data from these tables using a common column (usually a Primary Key and Foreign Key).

Joining multiple tables

You can join more than two tables:

SELECT s.student_name, c.course_name, t.teacher_name

FROM students s

JOIN courses c ON s.course_id = c.course_id

JOIN teachers t ON c.teacher_id = t.teacher_id;

Q-28.What is the GROUP BY clause in SQL? How is it used with aggregate functions?

A.The GROUP BY clause is used to group rows that have the same values in specified columns into summary rows.

Common Aggregate Functions

- COUNT() → counts rows
- SUM() → total value
- AVG() → average
- MAX() → highest value
- MIN() → lowest value

Example 1: COUNT with GROUP BY

SELECT course_name, COUNT(*) AS total_courses

FROM courses

GROUP BY course_name;

Counts how many records exist for each course.

Example 2: SUM with GROUP BY

```
SELECT course_name, SUM(course_duration) AS total_duration

FROM courses

GROUP BY course_name;
```

Calculates total duration per course.

Q-29.Explain the difference between GROUP BY and ORDER BY.

A.GROUP BY

Purpose:
Groups rows that have the same values so aggregate functions can be applied.

Used for:

- Summarizing data
- Counting per category
- Calculating totals/averages

Example:

```
SELECT course_name, COUNT(*) AS total

FROM courses

GROUP BY course_name;
```

Rows with the same course_name are grouped together.

ORDER BY

Purpose:
Sorts the result set in ascending or descending order.

Used for:

- Arranging output
- Ranking data
- Display formatting

Example:

```
SELECT * FROM courses

ORDER BY course_duration DESC;
```

Results are sorted, not grouped.

Q-30.What is a stored procedure in SQL, and how does it differ from a standard SQL query?

A.A stored procedure is a named set of SQL statements saved in the database and executed using a single call.

Basic Example

```
CREATE PROCEDURE getProducts()

BEGIN

  SELECT * FROM product;

END;
```

To execute:

```
CALL getProducts();
```

What is a Standard SQL Query?

A **standard SQL query** is a single SQL statement written and executed directly each time.

Example:

```
SELECT * FROM product;
```

Q-31.Explain the advantages of using stored procedures.

A.1. Improved Performance

2. Code Reusability

3. Enhanced Security

4. Reduced Network Traffic
5. Better Maintainability
6. Support for Complex Logic
7. Transaction Management

Q-32.What is a view in SQL, and how is it different from a table?

A.A view is a stored SQL query that presents data as a virtual table.

Key Differences Between View and Table

Feature	View	Table
Data storage	Does NOT store data	Stores data physically
Nature	Virtual table	Real table
Creation	Based on SELECT query	Created with CREATE TABLE
Space usage	Minimal (query only)	Uses disk space
Updatable	Sometimes limited	Fully updatable
Performance	May be slower	Usually faster

Security	Can hide columns/rows	Direct data access

Q-33.Explain the advantages of using viewsin SQL databases.

A.1.Enhanced Security

2. Simplifies Complex Queries

3. Data Abstraction

4. Reusability

5. Consistent Data Representation

6. No Extra Storage (for simple views)

7. Easier Maintenance

Q-34.What is a trigger in SQL? Describe its types and when they are used.

A.A trigger is a database object that runs automatically in response to INSERT, UPDATE, or DELETE events on a table.

Types of Triggers

1. BEFORE Trigger

Meaning:
Executes **before** the actual operation happens.

Used when:

- Validating data
- Modifying values before saving
- Preventing invalid entries

Example:

CREATE TRIGGER before_student_insert

```
BEFORE INSERT ON students

FOR EACH ROW

SET NEW.age = IF(NEW.age < 0, 0, NEW.age);
```

2. AFTER Trigger

Meaning:
Executes **after** the operation is completed.

Used when:

- Logging changes
- Auditing
- Updating related tables
- Sending notifications

Example:

```
CREATE TRIGGER after_course_delete

AFTER DELETE ON courses

FOR EACH ROW

INSERT INTO course_log(course_id)

VALUES (OLD.course_id);
```

Q-35.Explain the difference between INSERT, UPDATE, and DELETE triggers.

A.INSERT Trigger

When it fires:
When a new row is inserted into a table.

Purpose:

- Validate new data
- Auto-calculate values
- Maintain audit logs
- Enforce business rules

Example:

```
CREATE TRIGGER trg_after_insert

AFTER INSERT ON students

FOR EACH ROW

INSERT INTO student_log(student_id)

VALUES (NEW.student_id);
```

UPDATE Trigger

When it fires:
When an existing row is modified.

Purpose:

- Track changes
- Validate updates
- Maintain history
- Synchronize related tables

Example:

```
CREATE TRIGGER trg_after_update

AFTER UPDATE ON students

FOR EACH ROW

INSERT INTO student_audit(old_age, new_age)

VALUES (OLD.age, NEW.age);
```

DELETE Trigger

When it fires:
When a row is removed from a table.

Purpose:

- Keep deletion logs

- Maintain referential data
- Archive removed records

Example:

```
CREATE TRIGGER trg_after_delete

AFTER DELETE ON students

FOR EACH ROW

INSERT INTO deleted_students(student_id)

VALUES (OLD.student_id);
```

Q-36.What is PL/SQL, and how does it extend SQL's capabilities?

A.PL/SQL (Procedural Language/SQL) is a procedural extension of SQL developed by Oracle Corporation. It combines SQL's data manipulation power with procedural programming features like variables, loops, conditions, and exception handling.

How PL/SQL Extends SQL

1. Procedural Programming Support

SQL is declarative and cannot perform step-by-step logic. PL/SQL adds:

- Variables
- Loops (FOR, WHILE)
- Conditional statements (IF-THEN-ELSE)
- Blocks of code

Example (PL/SQL block):

```
DECLARE

 total NUMBER := 0;

BEGIN

 SELECT SUM(salary) INTO total FROM employees;

 DBMS_OUTPUT.PUT_LINE(total);
```

```
END;
```

SQL alone cannot handle such procedural flow.

2. Block Structure

PL/SQL programs are written in blocks:

```
DECLARE   -- optional (variables)
BEGIN     -- required (logic)
EXCEPTION -- optional (error handling)
END;
```

This makes programs organized and modular.

3. Exception Handling

PL/SQL can handle runtime errors gracefully.

```
BEGIN
 SELECT salary INTO v_sal FROM employees WHERE id = 10;
EXCEPTION
 WHEN NO_DATA_FOUND THEN
   DBMS_OUTPUT.PUT_LINE('No record found');
END;
```

SQL alone stops on error; PL/SQL can manage it.

4. Stored Program Units

PL/SQL enables creation of reusable database objects:

- Stored procedures
- Functions
- Packages
- Triggers

These improve performance and code reuse.

5. Better Performance

PL/SQL executes multiple SQL statements in one block, reducing network traffic between application and database.

6. Tight Integration with SQL

PL/SQL supports all SQL commands directly inside procedural code.

Example:

```
BEGIN
 UPDATE employees SET salary = salary * 1.1;
END;
```

Q-37.List and explain the benefits of using PL/SQL.

A.PL/SQL (Procedural Language/SQL) provides many advantages when working with databases, especially in systems like those from Oracle Corporation. It enhances SQL by adding programming features that make database operations more powerful and efficient.

Benefits of Using PL/SQL

1. Improved Performance

PL/SQL executes a block of SQL statements together on the database server.

Why it helps:

- Reduces network traffic between application and database
- Executes faster than sending multiple SQL statements separately

2. Procedural Capabilities

PL/SQL supports programming constructs such as:

- Variables
- Loops (FOR, WHILE)

- Conditional statements (IF-THEN-ELSE)

3. Exception Handling

PL/SQL provides built-in error handling.

Benefit:

- Prevents program crashes
- Allows graceful error recovery
- Makes applications more reliable

Example situations:

- NO_DATA_FOUND
- TOO_MANY_ROWS
- ZERO_DIVIDE

4. Code Reusability

You can create reusable program units:

- Stored procedures
- Functions
- Packages
- Triggers

5. Modularity

PL/SQL supports breaking programs into smaller logical blocks and packages.

Benefit:

- Cleaner code
- Easier debugging
- Better team development

6. Security

PL/SQL allows you to restrict direct access to tables.

Benefit:

- Users can execute procedures without accessing underlying data
- Helps enforce business rules
- Improves database security

7. Integration with SQL

PL/SQL works seamlessly with SQL statements like:

- SELECT
- INSERT
- UPDATE
- DELETE

8. Reduced Network Traffic

Instead of sending many SQL commands from the client, PL/SQL sends one block to the server.

Benefit:

- Faster execution
- Less network load
- Better scalability

9. Support for Large Applications

PL/SQL packages help organize enterprise-level applications.

Benefit: Suitable for complex, large database systems.

Q-38.What are control structures in PL/SQL? Explain the IF-THEN and LOOP control structures.

A.Control Structures in PL/SQL

Control structures in PL/SQL are statements that control the flow of execution in a program. They allow the program to make decisions and repeat actions based on conditions.

They mainly fall into two categories:

- Conditional control (decision making)
- Iterative control (looping)

IF-THEN Control Structure (Conditional)

The IF-THEN statement is used to execute a block of code only when a specified condition is TRUE.

Basic Syntax:

```
IF condition THEN

  statements;

END IF;
```

Example

```
DECLARE

  marks NUMBER := 75;

BEGIN

  IF marks >= 50 THEN

    DBMS_OUTPUT.PUT_LINE('Pass');

  END IF;

END;
```

LOOP Control Structure (Iterative)

LOOP is used to execute a set of statements repeatedly until a condition is met.

PL/SQL provides three main loop types:

- Simple LOOP
- WHILE LOOP
- FOR LOOP

1. Simple LOOP

Repeats indefinitely until exited using `EXIT` or `EXIT WHEN`.

Syntax

```
LOOP
  statements;
  EXIT WHEN condition;
END LOOP;
```

Example

```
DECLARE
  i NUMBER := 1;
BEGIN
  LOOP
    DBMS_OUTPUT.PUT_LINE(i);
    i := i + 1;
    EXIT WHEN i > 5;
  END LOOP;
END;
```

Output: 1 to 5

2. WHILE LOOP

Repeats **while condition is TRUE**.

Syntax

```
WHILE condition LOOP
```

```
  statements;
END LOOP;
```

Example

```
DECLARE
  i NUMBER := 1;
BEGIN
  WHILE i <= 5 LOOP
    DBMS_OUTPUT.PUT_LINE(i);
    i := i + 1;
  END LOOP;
END;
```

3. FOR LOOP

Used when the number of iterations is known.

Syntax

```
FOR counter IN start..end LOOP
  statements;
END LOOP;
```

Example

```
BEGIN
  FOR i IN 1..5 LOOP
    DBMS_OUTPUT.PUT_LINE(i);
  END LOOP;
END;
```

Q-39.How do control structures in PL/SQL help in writing complex queries?

A.1. Conditional Decision Making (IF–THEN)

Control structures let queries behave differently based on data values.

Use case: apply discounts only to certain customers.

```
BEGIN
  IF total_amount > 10000 THEN
     UPDATE orders
     SET discount = 0.10
     WHERE order_id = v_id;
  END IF;
END;
```

2. Row-by-Row Processing with Loops

Loops allow you to process multiple records sequentially when a single SQL statement is not enough.

Use case: update salaries department-wise.

```
FOR rec IN (SELECT emp_id, salary FROM employees) LOOP
  UPDATE employees
  SET salary = rec.salary * 1.1
  WHERE emp_id = rec.emp_id;
END LOOP;
```

3. Automating Repetitive Tasks

Instead of running the same query many times manually, loops automate repetition.

Examples:

- Generating monthly reports
- Bulk data cleanup
- Batch updates

4. Error Handling with EXCEPTION Blocks

Control structures allow safe handling of runtime errors.

```
BEGIN
  SELECT salary INTO v_sal FROM employees WHERE emp_id = 101;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Employee not found');
END;
```

5. Building Multi-Step Business Logic

Complex queries often require:

- intermediate calculations
- validations
- multiple dependent operations

Q-40.What is a cursor in PL/SQL? Explain the difference between implicit and explicit cursors.

A.A cursor in PL/SQL is a pointer that allows you to retrieve and process query result rows one at a time.

Implicit Cursor

An implicit cursor is automatically created by PL/SQL whenever you run SQL statements like:

- INSERT
- UPDATE
- DELETE

- SELECT INTO (single row)

You do not declare or control it manually.

Example

```
BEGIN
  UPDATE employees
  SET salary = salary + 1000
  WHERE emp_id = 101;
END;
```

Explicit Cursor

An **explicit cursor** is declared and controlled by the programmer to handle queries that return multiple rows.

You must manually:

1. Declare
2. Open
3. Fetch
4. Close

Example

```
DECLARE
  CURSOR emp_cursor IS
    SELECT emp_id, name FROM employees;
  v_id employees.emp_id%TYPE;
  v_name employees.name%TYPE;
BEGIN
  OPEN emp_cursor;
```

```
  LOOP
    FETCH emp_cursor INTO v_id, v_name;
    EXIT WHEN emp_cursor%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(v_id || ' ' || v_name);
  END LOOP;
  CLOSE emp_cursor;
END;
```

Q-41.When would you use an explicit cursor over an implicit one?

A. 1. When the query returns multiple rows

Implicit cursors (like SELECT INTO) work only for one row.
If your query returns many rows, you must use an explicit cursor.

Example use: processing all employees in a department.

2. When you need row-by-row processing

If each row needs separate logic, calculations, or conditional checks.

```
FOR rec IN emp_cursor LOOP
  -- custom logic per row
END LOOP;
```

3. When you need more control over fetching

Explicit cursors allow you to:

- open when needed
- fetch selectively
- stop early
- close manually

4. When implementing complex business logic

Use explicit cursors when operations depend on:

- previous row values
- conditional updates
- step-by-step processing
- batch operations

5. When using cursor attributes per cursor

Explicit cursors have their own attributes:

- cursor_name%FOUND
- cursor_name%ROWCOUNT
- etc.

Q-42.Explain the concept of SAVEPOINT in transaction management. How do ROLLBACK and COMMIT interact with savepoints?

A.Concept of SAVEPOINT in Transaction Management:

A SAVEPOINT in SQL is a marker set within a transaction that allows you to roll back part of the transaction instead of the whole transaction.

Interaction Between COMMIT and SAVEPOINT

COMMIT

- Makes **all changes permanent**
- Removes all savepoints
- Ends the transaction

Important: After COMMIT, you cannot roll back.

COMMIT;

All savepoints are erased.

ROLLBACK

There are two cases:

Full Rollback

ROLLBACK;

- Undoes entire transaction
- Removes all savepoints

Partial Rollback (to SAVEPOINT)

ROLLBACK TO sp1;

- Undoes changes **after sp1**
- Keeps earlier work
- Savepoints before sp1 remain

Q-43.When is it useful to use savepoints in a database transaction?

A.1. Complex multi-step transactions

When a transaction performs many related operations and some may fail.

Example: order processing

- create order
- add items
- update inventory
- process payment

If payment fails, you may want to undo only later steps.

2. Error handling in long transactions

In large transactions, fully rolling back can waste time and work. Savepoints allow partial recovery.

Benefit: avoids restarting the whole transaction.

3. Batch processing

When processing many records in one transaction.

Example:

- process 1000 rows
- if row 700 fails → rollback only that portion

This keeps earlier successful work.

4. Conditional business logic

When later operations depend on conditions.

You can:

- set savepoint
- try risky operation
- rollback if condition fails

5. Testing risky operations inside a transaction

Useful when trying updates that might violate constraints.

You can safely revert to the savepoint instead of losing everything.

6. Nested logical operations

When a transaction contains smaller logical units.

SAVEPOINT helps treat each unit like a mini-transaction.

Q-44.2₃₃₃|